

Homework 2 - Neural Network Dependency Parser

[Start Assignment](#)

- Due Wednesday by 11:59pm
- Points 100
- Submitting a file upload
- File Types zip, tgz, and tar.gz
- Available until Mar 15 at 11:59pm

Due: 11:59pm on Wednesday, March 11

Total Points: 100

You are welcome to discuss the problems with other students but you must turn in your own work. Please review the academic honesty policy for this course (at the end of the syllabus page).

Create a .zip or .tgz archive containing any of the files you submit. Upload that single file to Courseworks.

The file you submit should use the following naming convention:

YOURUNI_homework2.[zip | tgz]. For example, my uni is db2711, so my file should be named db2711_homework2.zip or db2711_homework2.tgz.

Introduction

The instructions below are fairly specific and it is okay to deviate from implementation details. **However: You will be graded based on the functionality of each function. Make sure the function signatures (function names, parameter and return types/data structures) match exactly the description in this assignment.**

Note that the training process without GPU acceleration may take about an hour or longer. Please allow for sufficient time before the due date. While the assignment is set up so that you can run it on your local environment (on a CPU), the training process can be sped up on a GPU.

Introduction

In this assignment you will train a feed-forward neural network to predict the transitions of an arc-standard dependency parser. The input to this network will be a representation of the current state (including tokens on the stack and buffer). The output will be a transition (shift, left_arc, right_arc), together with a dependency relation.

Much of the parser code is provided, but you will need to create the input representation for the neural net, decode the output of the network and apply the transitions, and also specify the network architecture and train the model.

As background reading for this project, consider taking a look at the following paper (this is not strictly required):

Chen, D., & Manning, C. (2014). [A fast and accurate dependency parser using neural networks](https://aclanthology.org/D14-1082.pdf)  (<https://aclanthology.org/D14-1082.pdf>). In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*.

Prerequisites: Installing necessary packages

We will use pytorch to build the neural network. pytorch is an open-source library for neural networks (and other mathematical models based on sequences of matrix and tensor computations), originally developed by Meta / Facebook AI, which has become the standard deep learning framework in NLP research.

You also need numpy for the feature extractor.

Installing pyTorch

We suggest you use the Python package management system pip.

On most systems, the following commands will work:

```
$ python -m pip install torch
```

Testing your Setup:

To test your setup, run a Python interpreter and type the following:

```
% python
Python 3.12.10 (main, Apr 12 2025, 06:28:14) [Clang 14.0.3 (clang-1403.0.22.14.1)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> torch.__version__
'2.10.0'
>>> import numpy
>>> numpy.__version__
'1.26.4'

(version numbers may differ)
```

Optionally, you can check if GPU support is available. For Nvidia GPUs using CUDA:

```
>>> torch.cuda.is_available()
False
```

Or on Apple Silicon chips (e.g. recent MacBook or MacBook Pro) using Metal Performance Shaders (MPS):

```
>>> torch.mps.is_available()
True
```

Getting Started

Please download the files and scaffolding needed for the dependency parser here [nn_dep_parser.zip](https://courseworks2.columbia.edu/courses/238524/files/26727161?wrap=1) (<https://courseworks2.columbia.edu/courses/238524/files/26727161?wrap=1>)  (https://courseworks2.columbia.edu/courses/238524/files/26727161/download?download_frd=1) (~7MB).

The archive contains the following python scripts:

- `conll_reader.py` - data structures to represent a dependency tree, as well as functionality to read and write trees in the CoNLL-X format (explained below).
- `get_vocab.py` - extract a set of words that appear in the training data. This is necessary to format the input to the neural net (the dimensionality of the input vectors depends on the number of words).
- `extract_training_data.py` - extracts two numpy matrices representing input output pairs (as described below). You will have to modify this file to change the input representation to the neural network.
- `train_model.py` - specify and train the neural network model. This script writes a file containing the model architecture and trained weights.
- `decoder.py` - uses the trained model file to parse some input. For simplicity, the input is a CoNLL-X formatted file, but the dependency structure in the file is ignored. Prints the parser output for each sentence in CoNLL-X format.
- `evaluate.py` - this works like `decoder.py`, but instead of ignoring the input dependencies it uses them to compare the parser output. Prints evaluation results.

You will only have to modify `extract_training_data.py`, `train_model.py` and `decoder.py` for this project.

There are also the following data files, corresponding to a standard split of the WSJ part of the Penn Treebank. The original Penn Treebank contains constituency parses, but these were converted automatically to dependencies.

- `data/train.conll` - Training data. ~40k sentences
- `data/dev.conll` - Development data. ~5k sentences. Use this to experiment and tune parameters.
- `data/sec0.conll` - section 0 of the Penn Treebank. ~2k sentence -- good for quick initial testing.
- `data/test.conll` - Test data. ~2.5k sentences. Don't touch this data until you are ready for a final test of your parser.

Dependency Format

The files are annotated using a modified CoNLL-X format (CoNLL is the conference on Computational Natural Language learning -- this format was first used for shared tasks at this conference). Each sentence corresponds to a number of lines, one per word. Sentences are separated with a blank line. You will need to be able to read these annotations and draw dependency trees (by hand) in order to debug your parser.

Each line contains fields, separated by a single tab symbol. The fields are, in order, as follows:

- word ID (starting at 1)
- word form

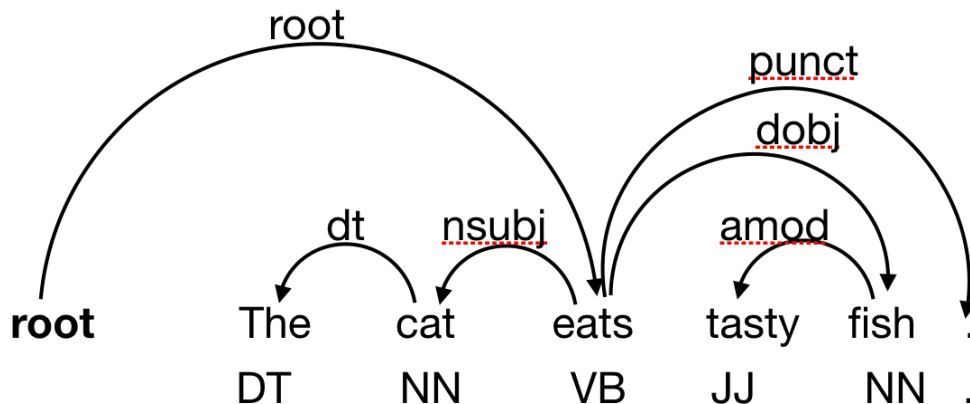
- lemma
- universal POS tag
- corpus-specific POS tag (for our purposes the two POS annotations are always the same)
- features (unused)
- word ID of the parent word ("head"). 0 if the word is the root of the dependency tree.
- dependency relation between the parent word and this word.
- deps (unused)
- misc annotations (unused)

Any field that contains no entry is replaced with a _.

For example, consider the following sentence annotation:

```
1 The _ DT DT _ 2 dt _ _
2 cat _ NN NN _ 3 nsubj _ _
3 eats _ VB VB _ 0 root _ _
4 tasty _ JJ JJ _ 5 amod _ _
5 fish _ NN NN _ 3 dobj _ _
6 . _ . . _ 3 punct _ _
```

The annotation corresponds to the following dependency tree



Take a look at data/sec0.conll for more examples.

The file conll_reader.py contains classes for representing dependency trees and reading in a CoNLL-X formatted data files. It is important that you understand how these data structures work, because you will use them to extract features below and also create dependency trees as parser output.

The class DependencyEdge represents a single word and its incoming dependency edge. It includes the attribute variables id, word, pos, head, deprel. Id is just the position of the word in the sentence. Word is the word form and pos is the part of speech. Head is the id of the parent word in the tree. Deprel is the dependency label on the edge pointing to this label. Note that the information in this class is a subset of what is represented in the CoNLL format.

The class `DependencyStructure` represents a complete dependency parse. The attribute `deprels` is a dictionary that maps integer word ids to `DependencyEdge` instances. The attribute `root` contains the integer id of the root node.

The method `print_conll` returns a string representation for the dependency structure formatted in CoNLL format (including line breaks).

Part 1 - Obtaining the Vocabulary (0 pts)

Because we will use one-hot representations for words in the input representation, we will need to know which words appear in the data, and we will need a mapping from words to indices.

Run the following

```
$python get_vocab.py data/train.conll data/words.vocab
```

to generate an index of types. This contains all words that appear more than once in the training data, plus special symbols. The words file will look like this (specific indices may differ):

```
<CD> 0
<NNP> 1
<UNK> 2
<ROOT> 3
<NULL> 4
blocking 5
hurricane 6
ships 7
```

The first 5 entries are special symbols. `<CD>` stands for any number (anything tagged with the POS tag CD), `<NNP>` stands for any proper name (anything tagged with the POS tag NNP). `<UNK>` stands for unknown words (in the training data, any word that appears only once). `<ROOT>` is a special root symbol (the word associated with token position 0, which is initially placed on the stack of the dependency parser). `<NULL>` is used to pad context windows.

Part 2 - Extracting Input/Output matrices for training (35 pts)

To train the neural network we first need to obtain a set of input/output training pairs. More specifically, each training example should be a pair (x,y) , where x is a parser state and y is the transition the parser should make in that state.

Take a look at the file `extract_training_data.py`

States: The input will be an instance of the class `State`, which represents a parser state. The attributes of this class consist of a stack, buffer, and partially built dependency structure `deps`. `stack` and `buffer` are lists of word ids (integers).

The top of the stack is the last word in the list `stack[-1]`. The next word on the buffer is also the last word in the list, `buffer[-1]`.

Deps is a list of (parent, child, relation) triples, where parent and child are integer ids and relation is a string (the dependency label).

Transitions: The output is a pair (transition, label), where the transition can be one of "shift", "left_arc", or "right_arc" and the label is a dependency label. If the transition is "shift", the dependency label is None. Since there are 45 dependency relations (see list `deps_relations`), there are $45 \times 2 + 1$ possible outputs.

Obtaining oracle transitions and a sequence of input/output examples.

As discussed in class, we cannot observe the transitions directly from the treebank. We only see the resulting dependency structures. We therefore need to convert the trees into a sequence of (state, transition) pairs that we use for training. *This part has already been implemented in the function `get_training_instances(dep_structure)`.* Given a `DependencyStructure` instance, this method returns a list of (State, Transition) pairs in the format described above.

TODO: Extracting Input Representations

Your task will be to convert the input/output pairs into a representation suitable for the neural network. You will complete the method `get_input_representation(self, words, pos, state)` in the class `FeatureExtractor`. The constructor of the class `FeatureExtractor` takes the vocabulary file as inputs (as a file object). It then stores a word-to-index dictionary in the attribute `word_vocab`.

`get_input_representation(self, words, pos, state)` takes as parameters a list of words in the input sentence, a list of POS tags in the input sentence and an instance of class `State`. It should return an encoding of the input to the neural network, i.e. a single vector.

To represent a state, we will use the top-three words on the buffer and the next-three word on the stack, i.e. `stack[-1]`, `stack[-2]`, `stack[-3]` and `buffer[-1]`, `buffer[-2]`, `buffer[-3]`. We could use embedded representations for each word, but we would like the network to learn these representations itself. Therefore, the neural network will contain an embedding layer and the words will be represented as a one-hot representation. The actual input will be the concatenation of the one-hot vectors for each word.

This would typically require a $6 \times |V|$ vector, but fortunately the keras embedding layer will accept integer indices as input and internally convert them. We therefore just need to return a vector (a 1-dimensional numpy array) of length 6.

So for example, if the next words on the buffer are "dog eats a" and "the" and `<ROOT>` are on the stack, the return value should be a numpy array `numpy.array([4047, 3, 4, 8346, 8995, 14774])`. Here 4 is the index for the `<NULL>` symbol, 3 is the index for the `<ROOT>` symbol, 4047 is the index for "the" and 8346, 8995, 14774 are the indices for "dog", "eats" and "a". (Your indices may differ from this example because the `get_vocab` script outputs a different index mapping each time it is run.)

Note that you need to account for the special symbols (`<CD>`, `<NNP>`, `<UNK>`, `<ROOT>`, `<NULL>`) in creating the input representation. This is why we pass the list of POS tags to the method. Make sure you take into account states in which there are less than 3 words on the stack or buffer.

This representation is a subset of the features in the Chen & Manning (2014) paper. Feel free to experiment with a more complete feature set once you got the basic version running.

TODO: Generating Input and Output matrices

Write the method `get_output_representation(self, output_pair)`, which should take a (transition, label) pair as its parameter and return a one-hot representation of these actions. Because there are $45 \times 2 + 1 = 91$ possible outputs, the output should be represented as a one-hot vector of length 91.

Saving training matrices

The neural network will take two matrices as its input, a matrix of training data (in the basic case a $N \times 6$ matrix, where N is the number of training instances) and an output matrix (an $N \times 91$ matrix).

The function `get_training_matrices(extractor, in_file)` will take a `FeatureExtractor` instance and a file object (a CoNLL formatted file) as its input. It will then extract state-transition sequences and call your input and output representation methods on each to obtain input and output vectors. Finally it will assemble the matrices and return them.

The main program in `extract_training_data.py` calls `get_training_matrices` to obtain the matrices and then writes them to two binary files (encoded in the numpy array binary format). You can call it like this:

```
python extract_training_data.py data/train.conll data/input_train.npy data/target_train.npy
```

You can also obtain matrices for the development set, which is useful to tune network parameters.

```
python extract_training_data.py data/dev.conll data/input_dev.npy data/target_dev.npy
```

Part 3 - Designing the network (30 pts)

Take a look at the file `train_model.py`.

Dataset and DataLoader

In pytorch we store training data in a `Dataset` class. The class `DependencyDataset` inherits from [torch.utils.data.Dataset](https://pytorch.org/docs/stable/data.html#torch.utils.data.Dataset)  and implements the following three methods:

`__init__(self, input_filename, target_filename)`: Instantiate the `DependencyDataset` class and load the data from the two `.npy` files you created in part 2.

`__len__(self)`: Return the total number of input/target pairs in the dataset.

`__getitem__(self, k)`: Return the input/target pair with index k .

pytorch also provides a `DataLoader` class that can be wrapped around a `Dataset` to easily use the dataset for training. The `DataLoader` allows us to easily adjust the batch size and shuffle the data. This will be used in the implementation of the training loop below.

Defining the neural network:

Now that we have training data, we can build the actual neural net. In pytorch, the conventional way to specify a neural network architecture is to write a class that inherits from [torch.nn.Module](https://pytorch.org/docs/stable/generated/torch.nn.Module.html)  (<https://pytorch.org/docs/stable/generated/torch.nn.Module.html>). Take a look at the documentation for this class.

A module can consist of multiple sub-modules, corresponding to layers of the neural network. These can be added in the `__init__` method of the new class as instance variables. Think of these as the parameters of the model that need to be trained. The class then has a method `forward(self, input)`, which defines the computation graph, i.e. how information is passed through the layers. Conceptually, the forward method is called on each batch and returns the output computed by the module.

Each module and sub-module can be called like a function, passing an input tensor and obtaining a result. For example `"embedded = self.embedding(inputs)"`.

TODO: Start by adding the following sub-modules to the init method:

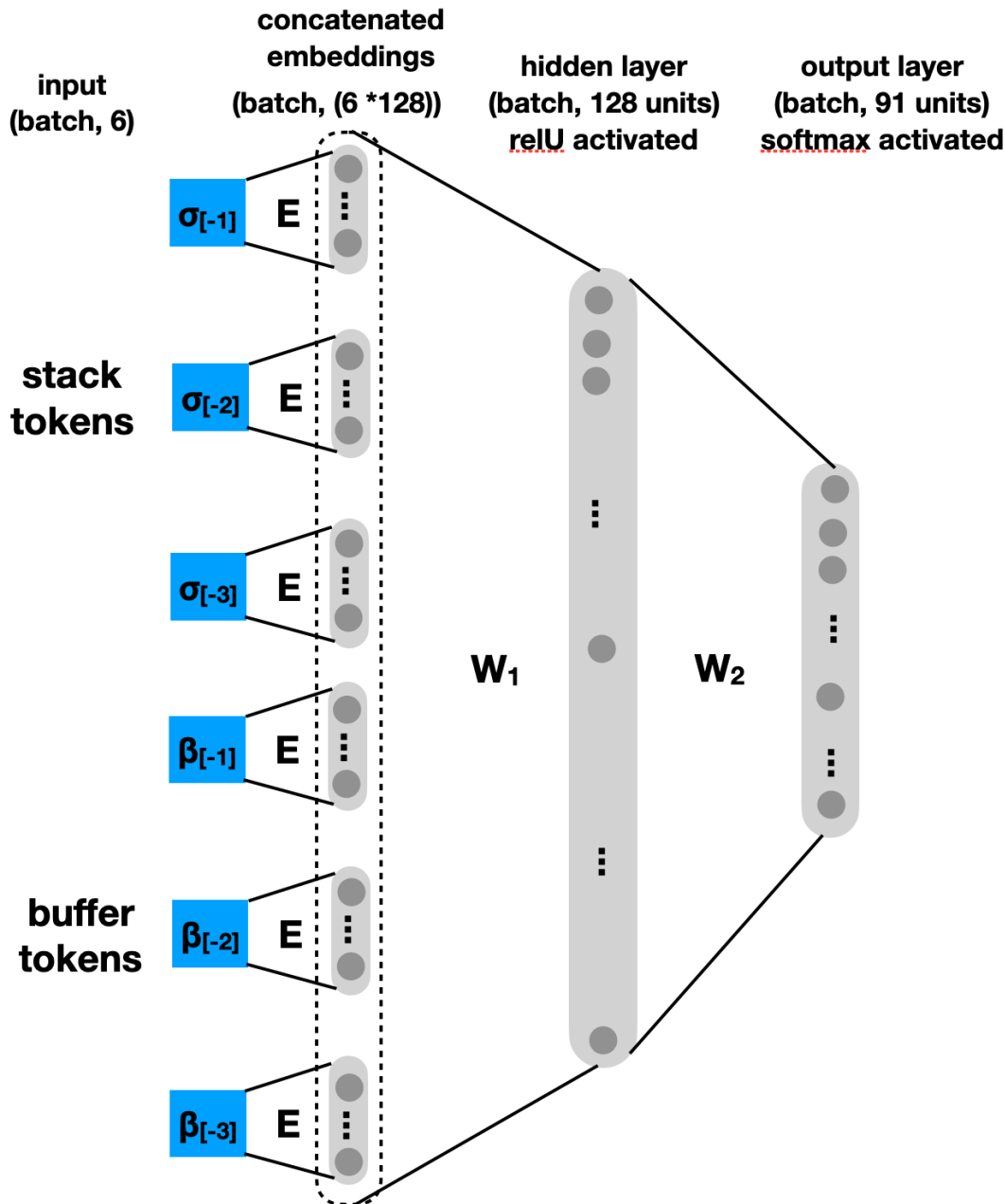
- One Embedding layer (i.e. an instance variable of type [torch.nn.Embedding](https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html)  (<https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>)). `num_embeddings` is the number of words_types (passed to the constructor). Set the `embedding_dim` (i.e. the number of units for each embedding) to 128.
- One [torch.nn.Linear](https://pytorch.org/docs/stable/generated/torch.nn.Linear.html)  (<https://pytorch.org/docs/stable/generated/torch.nn.Linear.html>) hidden layer of 128 units.
- One `torch.nn.Linear` output layer of 91 units.

Then, in the `forward(self, input)` method, define the computation graph as follows:

- obtain the embedded representation for each token by passing the *input* tensor to the embedding layer (i.e. `embedded = self.embedding(inputs)`). Note that the input is a (batch_size, 6) tensor and the output of the embedding layer will be a (batch_size, 6, 128) tensor. You will need to "flatten" this into a (batch_size, 768) tensor to pass to the hidden layer. This can be done using the `.view(...)`  (<https://pytorch.org/docs/stable/generated/torch.Tensor.view.html>) method on the tensor.
- Pass the concatenated embeddings (i.e. the (batch_size, 768) tensor) into the hidden layer. The result will be a (batch_size, 128) tensor. Apply the [torch.nn.functional.relu](https://pytorch.org/docs/stable/generated/torch.nn.functional.relu.html)  (<https://pytorch.org/docs/stable/generated/torch.nn.functional.relu.html>) activation function to this tensor (it will be applied pointwise).
- Then pass the result into the output layer. The result should be a (batch_size, 91) vector.
- Ordinarily we would apply the softmax function to the output to obtain a probability distribution over the 91 different transitions. But the loss function we will use ([CrossEntropyLoss](https://pytorch.org/docs/master/generated/torch.nn.CrossEntropyLoss.html)  (<https://pytorch.org/docs/master/generated/torch.nn.CrossEntropyLoss.html>)) implicitly computes the softmax for us. Instead, the `CrossEntropyLoss` function requires the raw logits.

For testing, it's a good idea to instantiate the `DependencyModel` class and then pass test inputs. I also find it useful to print the shapes of the intermediate representations while debugging.

The following figure illustrates the structure of the model:



Or as equations:

$$embeddings = [\sigma_{[-1]} \times E, \sigma_{[-2]} \times E, \sigma_{[-3]} \times E, \beta_{[-1]} \times E, \beta_{[-2]} \times E, \beta_{[-3]} \times E]$$

$$hidden = relu(embeddings \times W_1)$$

$$output = \log softmax(hidden \times W_2)$$

If using GPU acceleration, make sure to place all layers and data in the GPU memory using `[tensor].to('cuda')` or `[tensor].to('mps')`.

Part 4 - Training loop and running training (0 pts -- already built, but an important step)

Next, consider the function `train(model, loader)`, as well as the main section in `train_model.py`. The `train` function will train the model for a single epoch. It has already been completed for you.

The function first defines the loss function, which is 

<https://pytorch.org/docs/stable/generated/torch.nn.NLLLoss.html>) `CrossEntropyLoss` 

<https://pytorch.org/docs/master/generated/torch.nn.CrossEntropyLoss.html>). Then, it specifies the optimizer algorithm used for training. In class, we only discussed stochastic gradient descent. Here, we are using `Adagrad`  <https://pytorch.org/docs/stable/generated/torch.optim.Adagrad.html>) with a learning rate of 0.01.

The main loop iterates over the items obtained by the `DataLoader`. For each batch, it obtains the inputs (a (batch_size, 6) tensor) and the targets (a (batch_size, 91) tensor). It then calls the model to obtain the predictions for the inputs and computes the loss. Finally it performs the backward pass as follows:

```
optimizer.zero_grad() # set the gradient for all parameters to 0
loss.backward()       # recompute gradients based on current loss
optimizer.step()      # update the parameters based on the error gradients
```

The method also computes various statistics: it reports the training loss after each 1000 batches, as well as the accuracy after the epoch.

The main section at the bottom of `train_model.py` then runs the actual training. It first instantiates the model (passing the number of word types and number of output labels). Then, it loads the training data set, and wraps it in a `DataLoader` with a batch size of 16.

Then, it runs five epochs of training. In my experiments, after 5 epochs I reached a training loss of < 0.31 and a training accuracy of about 0.90.

Finally, the trained model parameters are saved to disk.

To run the training, call the following:

```
python train_model.py data/input_train.npy data/target_train.npy data/model.pt
```

Note that this may take an hour or longer if you train on a CPU, depending on your hardware.

Part 5 - Greedy Parsing Algorithm - Building and Evaluating the Parser (35 pts)

We will now use the trained model to construct a parser. In the file `decoder.py`, take a look at the class `Parser`. The class constructor takes the name of a pytorch file, loads the model and stores it in the instance variable `model`. It also uses the feature extractor from part 2.

TODO: Your task will be to write the method `parse_sentence(self, words, pos)`, which takes as parameters a list of words and POS tags in the input sentence. The method will return an instance of `DependencyStructure`.

The function first creates a `State` instance in the initial state, i.e. only word 0 is on the stack, the buffer contains all input words (or rather, their indices) and the `deps` structure is empty.

The algorithm is the standard transition-based algorithm discussed in class. As long as the buffer is not empty, we use the feature extractor to obtain a representation of the current state. We then call `model.predict(features)` and retrieve a softmax activated vector of possible actions.

In principle, we would only have to select the highest scoring transition and update the state accordingly. Unfortunately, it is possible that the highest scoring transition is not possible. `arc-left` or `arc-right` are not permitted the stack is empty. Shifting the only word out of the buffer is also illegal, unless the stack is empty. Finally, the root node must never be the target of a left-arc.

Instead of selecting the highest-scoring action, select the highest scoring *permitted* transition. The easiest way to do this is to create a list of possible actions and sort it according to their output probability (make sure the largest probability comes first in the list). Then go through the list until you find a legal transition.

The final step (which is already written for you) is to take the edge in `state.deps` and create a `DependencyStructure` object from it.

Running the `decoder.py` program like this should print CoNLL formatted parse trees for the sentences in the input (note, that the input contains dependency structures already, but these are ignored -- the output is generated by your parser).

```
python decoder.py data/model.pt data/dev.conll
```

To evaluate the parser, run the program `evaluate.py`, which will compare your parser output to the target dependency structures and compute labeled and unlabeled attachment accuracy.

```
python evaluate.py data/model.pt data/dev.conll
```

Labeled attachment score is the percentage of correct (parent, relation, child) predictions. Unlabeled attachment score is the percentage of correct (parent, child) predictions.

Even though the output looks reasonable for many sentences, the score for the parser is relatively low (~70 LAS). The current state of the art for dependency parsing is ~97 (

http://nlpprogress.com/english/dependency_parsing.html

(http://nlpprogress.com/english/dependency_parsing.html)). Feel free to experiment with additional features to improve the parser.

What you need to submit:

Please include

- all .py files
- your .vocab files
- the .pt model parameter file.

Do **not submit** the .npy matrices or the CoNLL data.